

REPORT DI LABORATORIO

Attacchi DoS (Denial of Service) Simulazione di un UDP Flood

Corso	Cybersecurity & Ethical Hacking
Anno Accademico	2025/2026
Studente	Lorenzo
Tipo attivita	Esercitazione di Laboratorio
Ambiente	Laboratorio Virtuale (Oracle VirtualBox)
Data	13 maggio 2026

Nota didattica: Questa esercitazione e stata svolta in un ambiente di laboratorio isolato e controllato, utilizzando macchine virtuali appositamente configurate per scopi formativi. L'obiettivo e esclusivamente didattico: comprendere il funzionamento di un attacco DoS di tipo UDP Flood per poter sviluppare contromisure adeguate. L'esecuzione di attacchi simili su sistemi reali senza autorizzazione esplicita e illegale e punibile penalmente.

1. Introduzione

Gli attacchi Denial of Service (DoS) rappresentano una delle minacce più diffuse e studiate nell'ambito della sicurezza informatica. Il loro obiettivo è semplice ma devastante: rendere un servizio, un server o un'intera rete inaccessibile agli utenti legittimi, sovraccaricando le risorse del sistema bersaglio con un volume di traffico o di richieste superiore a quello che il sistema stesso è in grado di gestire.

In questa esercitazione mi sono concentrato su una tipologia specifica di attacco DoS: l'UDP Flood. Il protocollo UDP (User Datagram Protocol) è un protocollo di trasporto connectionless, il che significa che non prevede l'instaurazione di una connessione prima dell'invio dei dati, a differenza del TCP che richiede un three-way handshake. Questa caratteristica rende UDP intrinsecamente più vulnerabile a forme di abuso, perché il target riceve pacchetti senza alcuna verifica preliminare sull'identità del mittente.

L'idea alla base dell'UDP Flood è semplice: inviare un numero massiccio di pacchetti UDP verso una o più porte della macchina vittima. Per ogni pacchetto ricevuto su una porta dove non è presente alcun servizio in ascolto, il sistema operativo genera una risposta ICMP "Destination Unreachable", consumando risorse di rete e di CPU. Quando il volume di pacchetti diventa sufficientemente elevato, il sistema non riesce più a gestire il traffico legittimo.

Per questa esercitazione ho scritto un semplice script Python che simula un UDP Flood, inviando pacchetti da 1 KB verso la porta 53 (DNS) della macchina Metasploitable 2.0. Ho monitorato l'intero processo sia dal lato attaccante con Wireshark, sia dal lato vittima con il comando `top` e `tcpdump`, per osservare l'impatto reale dell'attacco sulle risorse del sistema.

2. Obiettivi

1	Comprendere il funzionamento del protocollo UDP e la sua vulnerabilità agli attacchi di tipo flood.
2	Scrivere uno script Python in grado di generare e inviare un elevato numero di pacchetti UDP verso un target specifico.
3	Eseguire una ricognizione delle porte aperte sul target tramite Nmap, individuando una porta UDP adatta per l'attacco.
4	Monitorare il traffico di rete generato dall'attacco con Wireshark, analizzando protocollo, volume e direzione dei pacchetti.
5	Osservare l'impatto dell'attacco sulle risorse della macchina vittima (CPU, memoria, pacchetti persi) tramite top e tcpdump.
6	Riflettere sulle possibili contromisure e sull'importanza della prevenzione degli attacchi DoS.

3. Ambiente e Strumenti

Hypervisor	Oracle VirtualBox - Modalità Bridged Adapter
Attacker	Kali Linux 2026.1 - IP: 192.168.1.164
Target	Metasploitable 2.0 (Linux 2.6.24-16-server, i686) - IP: 192.168.1.170
Rete	LAN virtuale 192.168.1.0/24 - Gateway 192.168.1.1
Nmap	v7.99 - Scansione delle porte TCP/UDP sul target
Wireshark	Cattura e analisi del traffico di rete sull'interfaccia eth0
Script Python	UDP_flooder.py - Script personalizzato per la generazione del flood UDP
Python	v3.13.12 con virtual environment (venv_ai)

4. Procedura

STEP 1

Ricognizione del target con Nmap

Prima di procedere con l'attacco, ho effettuato una ricognizione della macchina target per identificare le porte aperte e i servizi attivi. Ho utilizzato Nmap con l'opzione `--top-ports 50` per scansionare le 50 porte TCP piu comuni sulla macchina Metasploitable (192.168.1.170).

Il comando eseguito e stato: `nmap --top-ports 50 192.168.1.170`

La scansione ha rivelato 12 porte aperte tra cui la porta 53/tcp (domain), che corrisponde al servizio DNS. Poiche il DNS utilizza comunemente anche il protocollo UDP, ho deciso di scegliere proprio la porta 53 come target per il mio UDP Flood. Tra le altre porte aperte figuravano FTP (21), SSH (22), Telnet (23), SMTP (25), HTTP (80), MySQL (3306) e VNC (5900), confermando l'elevata superficie di attacco di Metasploitable.

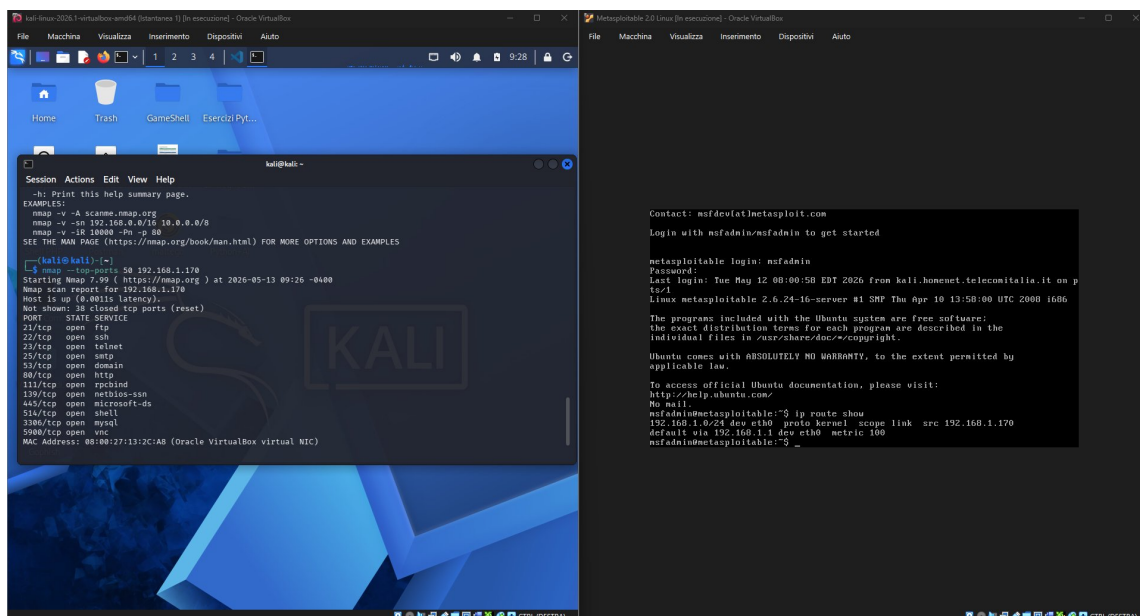


Fig. 1 - Scansione Nmap delle top 50 porte TCP su Metasploitable 2.0 e verifica della connettivita di rete dal lato target (ip route show).

STEP 2

Sviluppo dello script UDP_flooder.py

Ho scritto uno script Python chiamato `UDP_flooder.py` che implementa la logica dell'attacco UDP Flood. Lo script chiede in input l'indirizzo IP del target, la porta UDP di destinazione e il numero di pacchetti da inviare. Ogni pacchetto ha una dimensione di 1024 byte (1 KB) e contiene dati casuali generati con `random.randbytes(1024)`, in modo da produrre un payload non comprimibile.

Il socket viene creato con `socket.AF_INET` (IPv4) e `socket.SOCK_DGRAM` (UDP). L'invio avviene in un ciclo for, con un feedback visivo ogni 100 pacchetti per non rallentare l'esecuzione. Il codice completo e riportato nell'appendice del report.

STEP 3

Preparazione dell'ambiente e stato pre-attacco

Prima di lanciare l'attacco, ho preparato l'ambiente di monitoraggio. Sul lato Kali Linux ho aperto Wireshark impostando il filtro di cattura `ip.addr == 192.168.1.170` per isolare il traffico diretto verso la macchina target. Ho anche aperto lo script in Visual Studio Code e il terminale integrato per l'esecuzione.

Sul lato Metasploitable, ho eseguito il comando `top` per monitorare in tempo reale l'utilizzo delle risorse. In questa fase il sistema era stabile: load average a 3.09/3.43/2.11, CPU con 52.5% idle e 22.4% dedicato al sistema. La memoria mostrava 310588K utilizzati su circa 1 GB totale. Wireshark non registrava alcun traffico diretto verso il target.

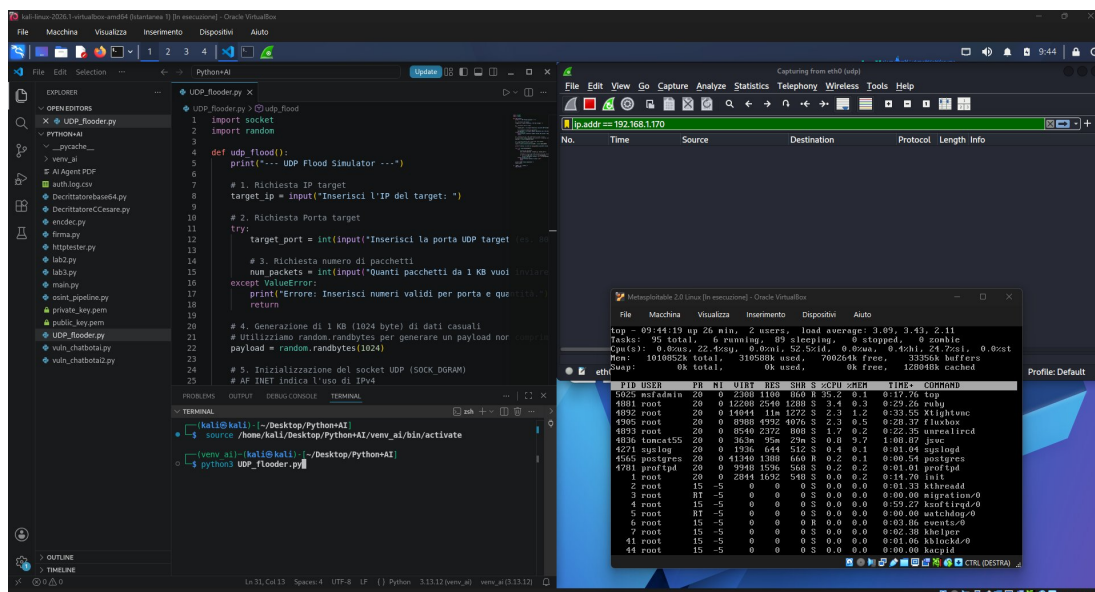


Fig. 2 - Stato pre-attacco: script aperto in VS Code, Wireshark in ascolto (nessun traffico), top su Metasploitable con carico nella norma.

STEP 4

Lancio dell'UDP Flood

Ho lanciato lo script dal terminale con `python3 UDP_flooder.py` e ho configurato i seguenti parametri: IP target 192.168.1.170, porta UDP 53. Per il primo tentativo ho impostato 1.000.000 di pacchetti da 1 KB ciascuno, per un volume totale teorico di circa 1 GB di dati.

Appena avviato lo script, Wireshark ha iniziato a registrare i primi pacchetti. Inizialmente i pacchetti catturati erano pochi, identificati come protocollo BROWSER (servizio di rete NetBIOS). Sul lato Metasploitable, il top ha mostrato un immediato cambiamento del carico: la CPU è passata a 69.0% nice e 14.0% system (valori che indicavano l'inizio dell'elaborazione del traffico in arrivo), con il load average che ha iniziato a salire a 2.23/2.72/2.11.

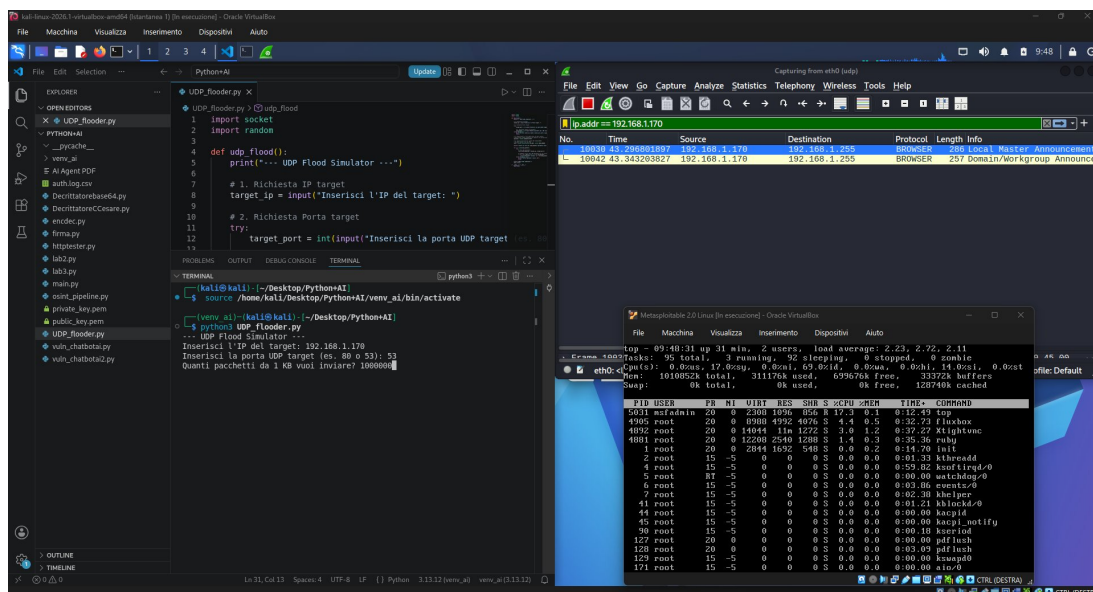


Fig. 3 - Avvio dell'UDP Flood: il terminale mostra l'inserimento dei parametri (IP 192.168.1.170, porta 53, 1.000.000 pacchetti). Wireshark registra i primi pacchetti. Il top su Metasploitable evidenzia l'aumento del carico CPU.

STEP 5

Monitoraggio durante l'attacco (secondo run - 10 milioni di pacchetti)

Dopo il primo test con 1.000.000 di pacchetti, ho deciso di ripetere l'attacco con un volume dieci volte superiore: 10.000.000 di pacchetti. A metà esecuzione circa (dopo l'invio di oltre 600.000 pacchetti su 10.000.000), la situazione era già molto diversa da quella iniziale. Wireshark aveva catturato circa 675.000 pacchetti, tutti classificati come protocollo DNS con dimensione 1066 byte e tipo "Unknown operation". Questo accade perché i pacchetti UDP inviati alla porta 53 vengono interpretati da Wireshark come query DNS, ma il contenuto (dati casuali) non corrisponde a una query DNS valida, da cui l'etichetta "Unknown operation".

Sul lato Metasploitable era visibile un intenso traffico DNS/MDNS: il sistema stava cercando di risolvere i pacchetti in arrivo, generando a sua volta risposte e query verso il DNS (`kali.homenet.telecomitalia.it` sulla porta 53275 verso 192.168.1.170). Erano visibili anche aggiornamenti MDNS (`updateMA`) che indicavano lo stress sulla componente di rete del sistema operativo.

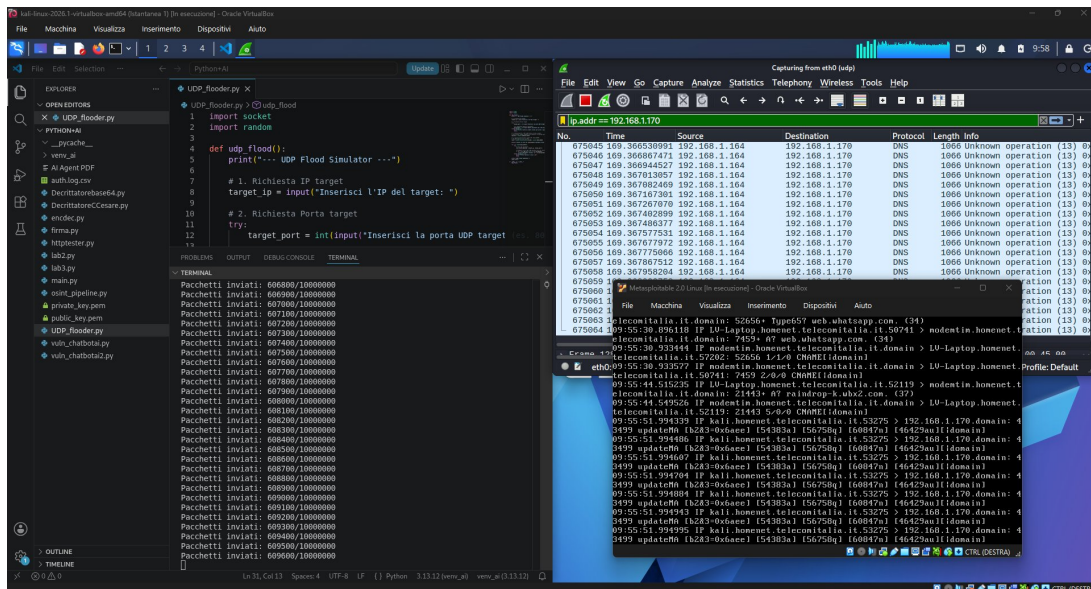


Fig. 4 - Durante l'attacco: il terminale riporta circa 609.000 pacchetti inviati. Wireshark mostra oltre 675.000 pacchetti DNS catturati. Sul target è visibile il traffico DNS e gli updateMA generati dallo stress di rete.

STEP 6

Analisi post-attacco e verifica con tcpdump

Al termine dell'invio di tutti i 10.000.000 di pacchetti, ho analizzato i risultati sul lato Metasploitable. Ho utilizzato tcpdump per catturare il traffico durante l'attacco e ho ottenuto dei dati molto significativi:

Su un totale di 286.745 pacchetti ricevuti dal filtro, ben 286.570 sono stati **droppati dal kernel**, cioè scartati dal sistema operativo perché non era in grado di elaborarli. Soltanto 61 pacchetti sono stati effettivamente catturati ed elaborati. Questo dato è estremamente significativo: il 99.94% dei pacchetti in arrivo è stato scartato, a dimostrazione del fatto che il sistema era completamente sovraccarico.

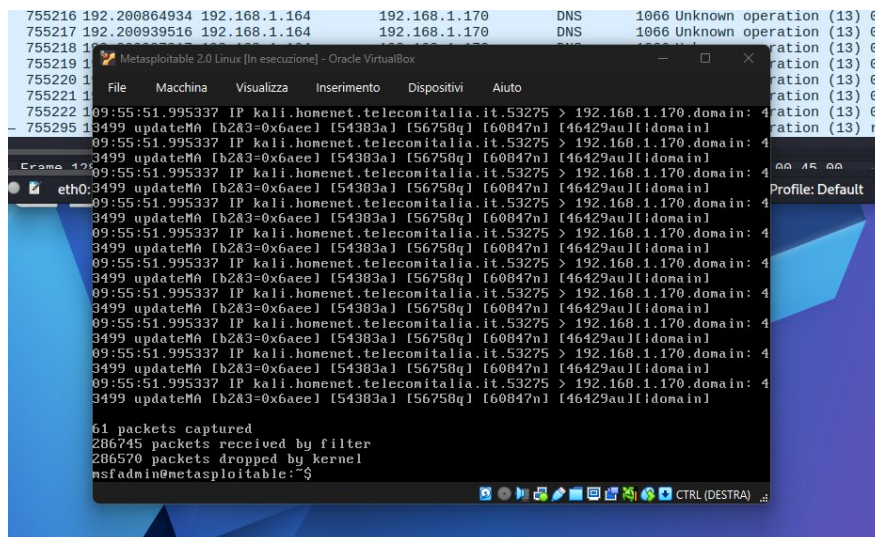


Fig. 5 - Output di tcpdump su Metasploitable al termine dell'attacco: 61 pacchetti catturati, 286.745 ricevuti dal filtro, 286.570 droppati dal kernel (99.94% di perdita).

5. Appendice - Codice Sorgente

UDP_flooder.py

```
import socket
import random

def udp_flood():
    print("--- UDP Flood Simulator ---")

    # 1. Richiesta IP target
    target_ip = input("Inserisci l'IP del target: ")

    # 2. Richiesta Porta target
    try:
        target_port = int(input("Inserisci la porta UDP target (es. 80 o 53): "))
    except ValueError:
        print("Errore: Inserisci numeri validi per porta e quantita.")
        return

    # 3. Richiesta numero di pacchetti
    num_packets = int(input("Quanti pacchetti da 1 KB vuoi inviare? "))

    # 4. Generazione di 1 KB (1024 byte) di dati casuali
    payload = random.randbytes(1024)

    # 5. Inizializzazione del socket UDP (SOCK_DGRAM)
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

    print(f"Invio in corso di {num_packets} pacchetti verso {target_ip}:{target_port}...")

    for i in range(num_packets):
        try:
            sock.sendto(payload, (target_ip, target_port))
            if (i + 1) % 100 == 0 or i + 1 == num_packets:
                print(f"Pacchetti inviati: {i + 1}/{num_packets}")
        except Exception as e:
            print(f"Errore durante l'invio: {e}")
            break

    print("UDP flood completato.")
    sock.close()

if __name__ == "__main__":
    udp_flood()
```


6. Risultati

✓ Script eseguito correttamente

Lo script UDP_flooder.py ha funzionato come previsto, inviando con successo prima 1.000.000 e poi 10.000.000 di pacchetti UDP da 1 KB ciascuno verso la porta 53 del target 192.168.1.170. L'esecuzione è stata completata senza errori e con feedback visivo regolare ogni 100 pacchetti.

✓ Traffico catturato e analizzato su Wireshark

Wireshark ha catturato centinaia di migliaia di pacchetti durante l'attacco, tutti classificati come DNS (porta 53) con dimensione 1066 byte. Il filtro `ip.addr == 192.168.1.170` ha permesso di isolare efficacemente il traffico relativo all'attacco.

✓ Impatto evidente sulla macchina vittima

L'analisi tramite `top` ha mostrato variazioni significative nel carico CPU e nel load average di Metasploitable durante l'attacco. Il dato più eclatante è quello fornito da `tcpdump`: su 286.745 pacchetti ricevuti, il 99.94% è stato scartato dal kernel, dimostrando il sovraccarico totale del sistema.

✓ Nessun crash del sistema target

Nonostante il volume elevato di traffico, Metasploitable non ha subito un crash completo. Il sistema ha continuato a funzionare, seppur in condizioni di forte degrado. Questo è coerente con la natura di un attacco DoS: l'obiettivo non è necessariamente mandare in crash il sistema, ma renderlo inutilizzabile per gli utenti legittimi.

7. Difficoltà Ricontrate

La prima difficoltà che ho incontrato è stata la scelta della porta da utilizzare per l'attacco. Inizialmente non ero sicuro se fosse meglio usare una porta dove c'era un servizio attivo (come la 53 per il DNS) oppure una porta casuale dove non c'era nulla in ascolto. Dopo aver ragionato un po', ho optato per la porta 53 perché volevo osservare come il servizio DNS della macchina vittima reagiva al flood, e perché il traffico sarebbe stato più "realistico" rispetto a un attacco su una porta vuota.

Un altro aspetto che mi ha fatto perdere un po' di tempo è stato il monitoraggio dal lato Metasploitable. All'inizio volevo usare solo `top`, ma mi sono reso conto che non bastava per avere un quadro completo dell'impatto, perché `top` mostra soprattutto il carico CPU e la memoria, non i pacchetti di rete. Ho quindi aggiunto `tcpdump` per verificare quanti pacchetti venivano effettivamente ricevuti e quanti venivano scartati dal kernel, e questo si è rivelato il dato più interessante dell'intera esercitazione.

Infine, all'inizio avevo impostato un numero di pacchetti troppo basso (qualche migliaio) e l'impatto sul target era quasi impercettibile. Ho dovuto aumentare il volume a 1.000.000 e oltre per vedere effetti realmente significativi, il che mi ha fatto capire quanto sia importante la scala in un attacco DoS: con pochi pacchetti il sistema assorbe il traffico senza problemi, ma appena si supera una certa soglia il degrado diventa evidente.

8. Conclusioni

Questa esercitazione mi ha permesso di toccare con mano il funzionamento e l'impatto di un attacco DoS di tipo UDP Flood. Ho potuto verificare che anche uno script relativamente semplice, scritto in poche righe di Python, e in grado di generare un volume di traffico sufficiente a mettere in difficoltà una macchina non protetta. Il dato chiave emerge dall'analisi post-attacco: con il 99.94% dei pacchetti scartati dal kernel, il sistema target era nella sostanza incapace di elaborare traffico legittimo.

Ho capito meglio anche il ruolo del protocollo UDP in questo tipo di attacchi. La mancanza di un meccanismo di handshake rende UDP il candidato ideale per i flood: l'attaccante può inviare pacchetti a velocità massima senza dover attendere alcuna risposta, e il target è costretto a processare ogni singolo pacchetto prima di poterlo scartare.

Dal punto di vista delle contromisure, questa esperienza mi ha fatto riflettere sull'importanza di strumenti come i firewall con rate limiting, gli IDS/IPS che rilevano pattern di traffico anomalo, e i sistemi di filtraggio a livello di rete (come le ACL sui router) che possono bloccare il traffico malevolo prima che raggiunga il server. In ambienti reali, soluzioni come il cloud-based DDoS protection (Cloudflare, AWS Shield) aggiungono un ulteriore livello di difesa distribuendo il carico su più nodi.

Report redatto nell'ambito del Corso di Cybersecurity & Ethical Hacking - Anno Accademico 2025/2026.